Studio Nx

Backend(FastAPI/Python) 62 — /api/v1/projects 라우터

2026-10-08

Backend(FastAPI/Python) 62 — 프로젝트 라우터 5개 (`/api/v1/projects`)

항목	내용
파일	`studionx/backend/app/api/v1/projects.py`
작업	P1-13 프로젝트 API — 주소 연결 · PRD 01 §7 · FR-122~126
작성	Claude (AI) · 2026-10-08 · 사용자 검토
이전 · 다음	61 `delete_project` · (P1-13 끝 — 개요 49, 다음 작업 P1-14 myProject.html JS)

1. 하는 일

요청 주소 5개를 서비스 함수(57~61)에 연결한다. 라우터는 **순서만 정하고**, 검사 · 저장은 서비스가 한다.

요청	성공	실패
`GET /api/v1/projects?query=&sort=&page=&size=`	200 `{items, total, page, size}`	401 로그인 안 함 · 422 틀린 값 (page 0, size 51, sort 모름, 검색어 101자)
`POST /api/v1/projects`	**201** `ProjectItem` (OWNER · 멤버 1 · opened false)	401 · 422 (56)
`GET /api/v1/projects/{id}`	200 `ProjectItem`	401 · **404** (없음 · 남의 것 · 삭제됨)
`PATCH /api/v1/projects/{id}`	200 `ProjectItem` (바뀐 값)	401 · 404 · **403** VIEWER · 422
`DELETE /api/v1/projects/{id}`	**204** 본문 없음	401 · 404 · **403** OWNER 아님

응답 예 (`POST`, 본문 `{"title": "My Song", "bpm": 128.5}`):

```
{"project_id": 12, "title": "My Song", "description": null, "bpm": "128.50",
 "time_sig_num": 4, "time_sig_den": 4, "music_key": null,
 "created_at": "2026-10-08T02:10:11.123000", "updated_at": "2026-10-08T02:10:11.123000",
 "my_role": "OWNER", "member_count": 1, "opened": false}
```

`bpm` 은 **글자**(`"128.50"`)로 온다 — 소수(Decimal)를 정확히 보내려는 pydantic 규칙. 화면(P1-14)에서 숫자가 필요하면 `Number(item.bpm)`.

2. 언제 쓰나

화면 (P1-14 myProject.html)	요청
페이지 열기 · 검색 · 정렬 · 페이지 버튼	`GET /projects`
새 프로젝트 모달 Create	`POST /projects` → 넓은 화면이면 시퀀서 창 열기 (FR-126 · 129)
카드 Edit details Save	`PATCH /projects/{id}`
카드 Delete (+ `confirmDialog`)	`DELETE /projects/{id}`
시퀀서가 열 때 제목 표시 (Phase 2)	`GET /projects/{id}`

`CurrentUser` 는 토큰을 검사해 회원을 넘겨주는 의존성(P1-03). 토큰이 없거나 만료되면 이 함수들은 불리지도 않고 401.

3. 코드

import (파일 맨 위)

```
from typing import Annotated

from fastapi import APIRouter, Query, status

from app.api.deps import CurrentUser, Db
from app.schemas.common import ErrorResponse
from app.schemas.project import (
    PROJECT_PAGE_SIZE_MAX,
    ProjectCreateRequest,
    ProjectItem,
    ProjectListResponse,
    ProjectSort,
    ProjectUpdateRequest,
)
from app.services import project_service
```

`from app.services import project_service` 줄을 이번에 추가했다. 나머지는 템플릿 그대로 — 주소 · 요청 · 응답 모양(`Query` · 스키마 · `status`)을 정하는 데 쓴다.

함수 (설명 주석 생략)

```
router = APIRouter()

NOT_FOUND = {404: {"model": ErrorResponse}}
FORBIDDEN = {403: {"model": ErrorResponse}}


@router.get("", response_model=ProjectListResponse)
def list_projects(
    user: CurrentUser,
    db: Db,
    query: Annotated[str, Query(max_length=100)] = "",
    sort: ProjectSort = ProjectSort.UPDATED,
    page: Annotated[int, Query(ge=1)] = 1,
    size: Annotated[int, Query(ge=1, le=PROJECT_PAGE_SIZE_MAX)] = 10,
) -> ProjectListResponse:
    return project_service.list_projects(db, user, query.strip(), sort, page, size)


@router.post("", response_model=ProjectItem, status_code=status.HTTP_201_CREATED)
def create_project(data: ProjectCreateRequest, user: CurrentUser, db: Db) -> ProjectItem:
    project = project_service.create_project(db, user, data)
    # 방금 만든 프로젝트라 멤버는 나 하나 · 연 적 없음 → DB 에 다시 묻지 않음
    return project_service.to_item(db, project, project.members[0], member_count=1, opened=False)


@router.get("/{project_id}", response_model=ProjectItem, responses=NOT_FOUND)
```

```
def read_project(project_id: int, user: CurrentUser, db: Db) -> ProjectItem:
    project, member = project_service.get_membership(db, project_id, user)
    return project_service.to_item(db, project, member)


@router.patch(
    "/{project_id}",
    response_model=ProjectItem,
    responses={**NOT_FOUND, **FORBIDDEN},
)
def update_project(
    project_id: int,
    data: ProjectUpdateRequest,
    user: CurrentUser,
    db: Db,
) -> ProjectItem:
    project, member = project_service.get_membership(db, project_id, user)
    project = project_service.update_project(db, project, member, data)
    return project_service.to_item(db, project, member)


@router.delete(
    "/{project_id}",
    status_code=status.HTTP_204_NO_CONTENT,
    responses={**NOT_FOUND, **FORBIDDEN},
)
def delete_project(project_id: int, user: CurrentUser, db: Db) -> None:
    project, member = project_service.get_membership(db, project_id, user)
    project_service.delete_project(db, project, member)
```

문법 풀이

import 문법	뜻
`from app.services import project_service`	모듈째 가져옴 → `project_service.create_project` 처럼 씀. 라우터 함수 이름(`create_project`)과 서비스 함수 이름이 같아도 섞이지 않음

식	뜻
`@router.get("", response_model=...)`	이 함수 = `GET /api/v1/projects`. 돌려준 값을 응답 스키마 모양으로 걸러 JSON 으로
`status_code=status.HTTP_201_CREATED`	성공 코드 201 (새로 만듦). 삭제는 204 (본문 없음)
`responses={**NOT_FOUND, **FORBIDDEN}`	API 문서(/docs)에 404 · 403 응답 모양을 적음. 동작에는 영향 없음
`Annotated[int, Query(ge=1)] = 1`	주소의 `?page=` 값, 1 이상, 없으면 1. 틀리면 FastAPI 가 422
`query.strip()`	검색어 앞뒤 공백 제거 — `" gam "` → `"gam"`, `" "` → `""`(전체)
`project, member = ...get_membership(...)`	함수가 돌려준 두 값을 두 변수에 나눠 받음
`project.members[0]`	관계(P1-03)로 이 프로젝트의 멤버 목록 → 방금 넣은 OWNER 행 하나

4. 흐름

요청	순서
목록	검색어 공백 제거 → `list_projects` (58)
만들기	`create_project` (59) → `to_item` 에 멤버 1 · opened False 를 넘김
상세	`get_membership` (57) → `to_item`
수정	`get_membership` → `update_project` (60) → `to_item`
삭제	`get_membership` → `delete_project` (61) → 204

어느 단계에서든 `AppError` 가 나면 처리기(P1-03)가 `{"error": {"code", "message", "details"}}` 로 응답한다.

5. 왜 이렇게

선택	이유
라우터는 연결만	권한 · 검사 · 저장은 서비스에 — 서비스는 HTTP 없이 시험할 수 있고(57~61), 시퀀서 API(Phase 2)도 같은 함수를 씀
상세 · 수정 · 삭제 모두 `get_membership` 먼저	남의 프로젝트는 역할 검사 전에 404 로 끝남 (57 결정 A)
만들기 응답에 값 직접 넘김	방금 만든 프로젝트의 멤버 수(1) · opened(False)는 확실하므로 쿼리 두 번을 아낌 (템플릿)
수정 · 상세도 `ProjectItem`	목록 한 줄과 같은 모양 → 화면이 그 카드만 다시 그리면 됨
삭제는 204	돌려줄 내용이 없음. 화면은 성공이면 카드를 지우고 목록을 다시 읽음

6. 시험 (`tests/test_project_routes.py` · 12개 추가, 모두 13개 — 실제 앱 + MySQL)

시험	확인한 것
만들기	201, 응답 칸 12개, 제목 공백 정리, bpm `"128.50"`, 7/8, OWNER · 1 · false, DB 주인
만들기 422	공백 제목 → `VALIDATION_ERROR`
로그인 안 함	목록 · 만들기 · 상세 → 401
목록	이름순 2개씩 → `alpha` · `Beta`, total 3 / 검색 `" gam "` → `Gamma`
목록 422 (5가지)	page 0 · size 0 · size 51 · sort random · 검색어 101자
상세	내 것 200 (만들기 응답과 같음), 남의 것 · 없는 번호 404 "Project not found."
수정	OWNER 제목 → 설명 그대로 · 멤버 3, EDITOR 설명 지움, VIEWER 403, 제목 null 422, DB 그대로
삭제	EDITOR 403 → OWNER 204 본문 없음 → 상세 · 다시 삭제 404, 목록 total 0

전체 시험 492개 통과, `ruff` 검사 통과.